Relatório Trabalho - Sistema Centro Esportivo

Banco de Dados 2 · Prof. Raul E. S. Castanhari · IFSP Jacareí · ADS · 2026

Integrantes: Camila R. Ribeiro, Helena Vieira Nascimento e Maria Clara Pastor

Repositório dos códigos: https://github.com/camilaribeirox/centro-esportivo

1. Diagrama Entidade-Relacionamento Conceitual

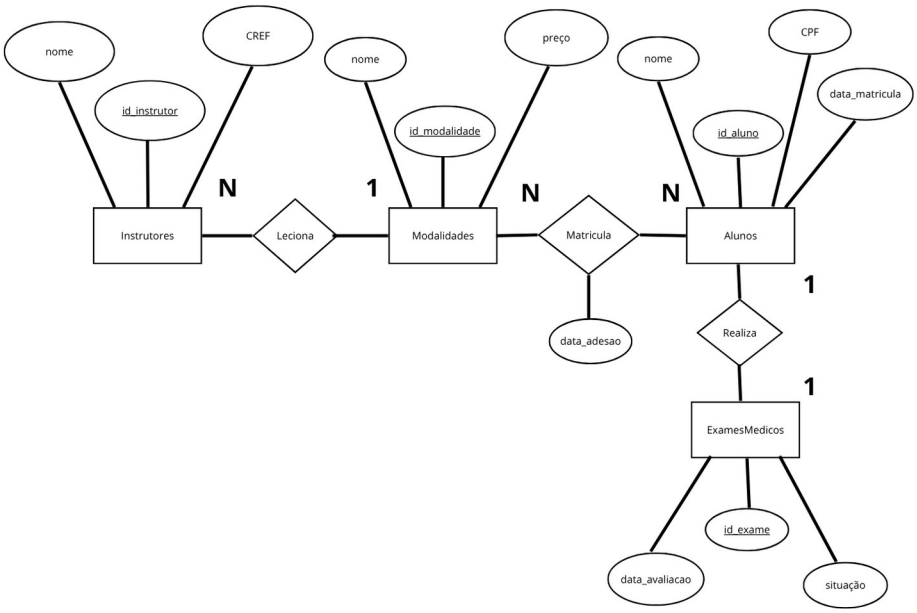


Figura 1: DER conceitual do centro esportivo

É a representação gráfica que descreve o quê o sistema precisa guardar da realidade, sem se preocupar com comandos técnicos, SQL ou qual SGBD será usado. É o diagrama feito para o cliente e analista validarem as regras de negócio.

Entidades: Representadas por retângulos (Instrutores, Modalidades, Alunos, ExamesMedicos).

Relacionamentos: Representados por losangos (Leciona, Matricula, Realiza).

Atributos: Representados por elipses ligadas às entidades, com chaves identificadoras devidamente sublinhadas (id_instrutor, id_modalidade, id_aluno, id_exame).

Atributo de Relacionamento: O atributo data_adesao está associado diretamente ao losango Matricula (relacionamento N:N com atributos próprios).

Posicionamento de Cardinalidades: "A cardinalidade de uma entidade é sempre colocada na linha após o losango":

- Instrutores (N) -- [Leciona] -- (1) Modalidades: 1 instrutor leciona 1 modalidade; 1 modalidade pode reunir N instrutores.

- Modalidades (N) -- [Matricula] -- (N) Alunos: 1 modalidade reúne N alunos; 1 aluno pode cursar N modalidades.
- Alunos (1) -- [Realiza] -- (1) ExamesMedicos: 1 aluno realiza 1 exame médico cadastrado; 1 exame pertence a 1 aluno.

2. Modelo Lógico Relacional

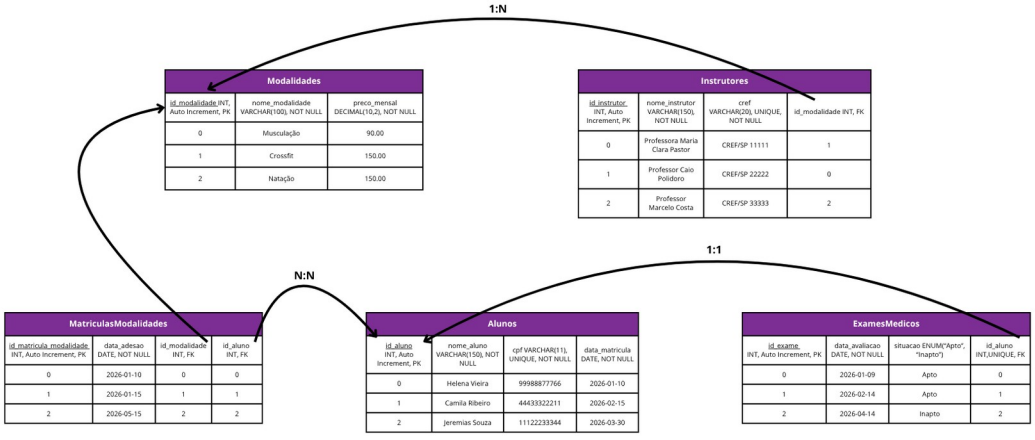


Figura 2: modelo lógico relacional com dados de teste

É a representação técnica direta de como os dados serão organizados no SGBD Relacional (MySQL). Ele traduz as entidades do DER em tabelas bidimensionais com colunas e linhas.

Tabelas e Tipos: Todas as 5 tabelas possuem suas colunas tipadas (INT, VARCHAR, DECIMAL, DATE, ENUM), com PKs com Auto Increment e chaves identificadas.

Dados de Teste: Tuplas reais de teste (ex: Professora Maria Clara Pastor, Helena Vieira, Jeremias Souza, etc.).

Relação 1:N (Modalidades e Instrutores): A tabela do lado N (Instrutores) contém a Chave Estrangeira id_modalidade INT, FK apontando para Modalidades.

Relação N:N (Alunos e Modalidades): Tabela associativa MatriculasModalidades contendo as Chaves Estrangeiras id_modalidade e id_aluno e o atributo da adesão data_adesao DATE.

Relação 1:1 (Alunos e ExamesMedicos): Adicionamos id_aluno INT, UNIQUE, FK na tabela ExamesMedicos. A restrição UNIQUE é o requisito essencial para garantir que a relação seja estritamente 1 para 1.

Setas de Integridade Referencial: Indicam perfeitamente a chave de origem apontando para a chave primária de destino.

A tabela abaixo resume as chaves e restrições implementadas no script centro_esportivo.sql:

Tabela	Chave primária	Chaves estrangeiras	Restrições
Modalidades	id_modalidade	-	CHECK (preco_mensal >= 0)
Alunos	id_aluno	-	UNIQUE (cpf)
Instrutores	id_instrutor	id_modalidade → Modalidades	UNIQUE (cref), ON DELETE RESTRICT
ExamesMedicos	id_exame	id_aluno → Alunos	UNIQUE (id_aluno), situacao Apto/Inapto, ON DELETE CASCADE
MatriculasModalidades	id_matricula_modalidade	id_modalidade, id_aluno	UNIQUE (id_aluno, id_modalidade), ON DELETE CASCADE

3. Diagrama de Classes de Entidades

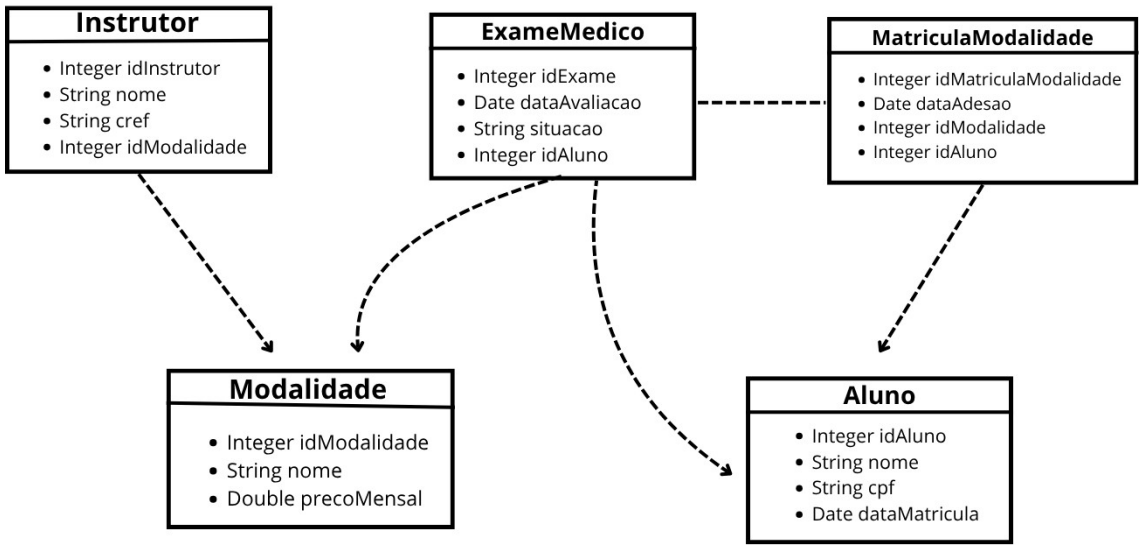


Figura 3: diagrama de classes de entidades

É um diagrama que modela como as entidades do banco de dados são representadas como Classes na linguagem de programação (em Java, C#, Python, etc.) dentro da memória da aplicação.

Classes de domínio: Instrutor, Modalidade, ExameMedico, MatriculaModalidade, Aluno.

Mapeamento de relacionamentos via identificador: Os relacionamentos entre as classes são feitos pelo identificador do objeto relacionado (Integer idModalidade, Integer idAluno), e não por referência direta ao objeto. As setas tracejadas indicam essa dependência: a classe que guarda o id aponta para a classe referenciada.

Cada classe corresponde a uma tabela do modelo relacional, e cada atributo corresponde a uma coluna:

Classe (app.py)	Atributos	Tabela no banco	Relação
Modalidade	idModalidade, nome, precoMensal	Modalidades	Referenciada por Instrutor e MatriculaModalidade
Instrutor	idInstrutor, nome, cref, idModalidade	Instrutores	1:N com Modalidade (guarda idModalidade)
Aluno	idAluno, nome, cpf, dataMatricula	Alunos	Referenciada por MatriculaModalidade e ExameMedico
MatriculaModalidade	idMatriculaModalidade, dataAdesao, idModalidade, idAluno	MatriculasModalidades	N:N entre Aluno e Modalidade
ExameMedico	idExame, dataAvaliacao, situacao, idAluno	ExamesMedicos	1:1 com Aluno (guarda idAluno)

No código Python, as classes ficam no início do arquivo app.py. Cada uma tem o método to_dict(), usado pelo adaptador para transformar o objeto em registro.

4. Diagrama de Arquitetura de Persistência

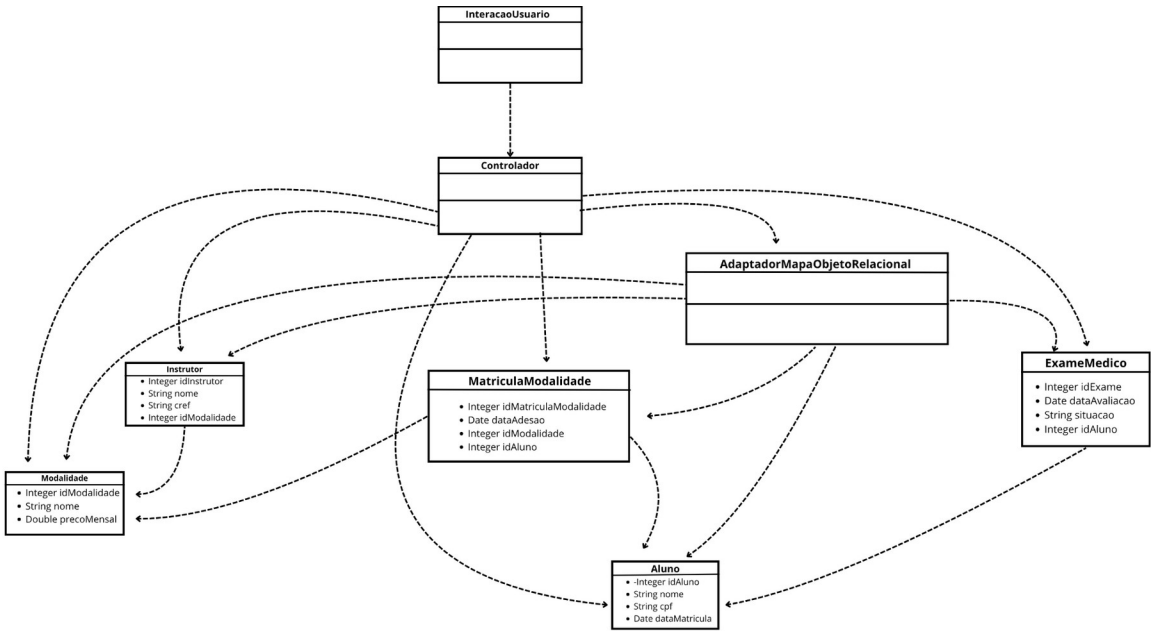


Figura 4: arquitetura de persistência com adaptador objeto-relacional

É o diagrama que mostra como o sistema inteiro se comunica para salvar e recuperar os dados no banco sem bagunçar o código, eliminando a incompatibilidade de impedância (a diferença entre objetos em memória e tabelas no banco).

O fluxo demonstra:

- InteracaoUsuario aciona o Controlador.
- O Controlador manipula os objetos de negócio (Modalidade, Instrutor, MatriculaModalidade, Aluno, ExameMedico).
- O AdaptadorMapaObjetoRelacional é o componente único responsável por serializar/desserializar os objetos e gerar dinamicamente as transações SQL junto ao SGBD.

No código, cada caixa do diagrama é uma classe do arquivo app.py:

Componente do diagrama	Classe em app.py	Responsabilidade
InteracaoUsuario	InteracaoUsuario	Menus no terminal, leitura dos dados digitados e exibição das tabelas
Controlador	ControladorCentroEsportivo	Regras de negócio: valida CPF, CREF, datas e evita matrícula duplicada
Classes de entidade	Modalidade, Instrutor, Aluno, MatriculaModalidade, ExameMedico	Guardam os dados em memória
AdaptadorMapaObjetoRelacional	AdaptadorMapaObjetoRelacional	Única classe que executa SQL: converte objetos em registros e registros em objetos

5. Implementação do código

Todo o código fonte está no repositório: https://github.com/camilaribeirox/centro-esportivo

O repositório contém:

- **centro_esportivo.sql:** script completo em MySQL 8 com a criação do banco, das 5 tabelas, a carga de dados (INSERT), as 4 seleções complexas, as atualizações (UPDATE) e as remoções (DELETE).
- **app.py:** aplicação em Python, no terminal, que implementa a arquitetura da Figura 4. Usa apenas a biblioteca padrão (sqlite3), então roda com o comando python app.py, sem instalar nada.
- **diagramas/:** as imagens das Figuras 1 a 4.
- **README.md:** instruções de execução e resumo do projeto.

A aplicação oferece seleção, inserção, atualização e remoção para todas as tabelas:

Tabela	Seleção	Inserção	Atualização	Remoção
Modalidades	Listar	Cadastrar	Editar nome e preço	Excluir (bloqueado se houver instrutor)
Instrutores	Listar com modalidade	Cadastrar	Editar nome, CREF e modalidade	Excluir
Alunos	Listar	Cadastrar	Editar nome, CPF e	Excluir (apaga

			data	matrículas e exame em cascata)
MatriculasModalidades	Listar com valores	Matricular aluno	-	Cancelar matrícula
ExamesMedicos	Listar	Lançar laudo	Atualizar laudo	Remover laudo

5.1 Criação das tabelas

As tabelas sem chave estrangeira (Modalidades e Alunos) são criadas primeiro. Em seguida vêm as tabelas que dependem delas. O exemplo abaixo mostra a relação 1:N, em que a chave estrangeira fica no lado N:

```
CREATE TABLE Instrutores (
    id_instrutor INT AUTO_INCREMENT PRIMARY KEY,
    nome_instrutor VARCHAR(150) NOT NULL,
    cref VARCHAR(20) NOT NULL UNIQUE,
    id_modalidade INT NOT NULL,
    CONSTRAINT fk_instrutor_modalidade
        FOREIGN KEY (id_modalidade) REFERENCES Modalidades(id_modalidade)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
) ENGINE=InnoDB;
```

5.2 Inserção (INSERT)

A carga inicial insere 27 registros: 5 modalidades, 5 alunos, 5 instrutores, 5 exames e 7 matrículas. As tabelas pai são preenchidas antes das que têm chave estrangeira.

```
INSERT INTO Modalidades (id_modalidade, nome_modalidade, preco_mensal) VALUES
(1, 'Musculação', 90.00),
(2, 'Crossfit', 150.00),
(3, 'Natação', 150.00),
(4, 'Pilates', 130.00),
(5, 'Boxe', 110.00);
```

Na aplicação, a inserção é feita pelo adaptador, que recebe um objeto e gera o comando SQL. O id gerado pelo banco é devolvido ao objeto:

```
def inserir_modalidade(self, modalidade: Modalidade) -> int:
    with self.get_connection() as conn:
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO Modalidades (nome_modalidade, preco_mensal) VALUES (?, ?);",
            (modalidade.nome_modalidade, modalidade.preco_mensal)
        )
        conn.commit()
        return cur.lastrowid
```

5.3 Seleção simples (SELECT)

A seleção faz o caminho inverso: cada linha da tabela é convertida em um objeto da classe correspondente (desserialização).

```
def listar_alunos(self) -> List[Aluno]:
    with self.get_connection() as conn:
        cur = conn.cursor()
        cur.execute("SELECT id_aluno, nome_aluno, cpf, data_matricula FROM Alunos ORDER BY
id_aluno ASC;")
        rows = cur.fetchall()
        return [Aluno(r["id_aluno"], r["nome_aluno"], r["cpf"], r["data_matricula"]) for r
in rows]
```

5.4 Atualização (UPDATE)

Foram implementados três tipos de atualização: em lote (várias linhas por uma condição), por id e por CPF.

```
-- 1. Reajuste de 10% nas mensalidades superiores a R$ 100,00
UPDATE Modalidades
SET preco_mensal = ROUND(preco_mensal * 1.10, 2)
WHERE preco_mensal > 100.00;

-- 2. Liberação médica do aluno Jeremias Souza (id 3)
UPDATE ExamesMedicos
SET situacao = 'Apto', data_avaliacao = '2026-06-01'
WHERE id_aluno = 3;

-- 3. Atualização cadastral da aluna Helena Vieira pelo CPF
UPDATE Alunos
SET nome_aluno = 'Helena Vieira da Silva'
WHERE cpf = '99988877766';
```

Comando	Efeito
Reajuste de 10%	4 modalidades acima de R$ 100,00 reajustadas: Crossfit e Natação vão a R$ 165,00, Pilates a R$ 143,00 e Boxe a R$ 121,00
Liberação médica	O exame de Jeremias Souza passa de Inapto para Apto
Atualização cadastral	A aluna localizada pelo CPF 99988877766 passa a se chamar Helena Vieira da Silva

5.5 Remoção (DELETE)

A remoção mostra o funcionamento das regras de integridade referencial definidas nas chaves estrangeiras.

```
-- 1. Cancelamento da matrícula de Jeremias Souza em Natação
DELETE FROM MatriculasModalidades
WHERE id_aluno = 3 AND id_modalidade = 3;
```

```
-- 2. Remoção da aluna Camila Ribeiro (id 2) e teste de exclusão em cascata
DELETE FROM Alunos
WHERE id_aluno = 2;

-- 3. Conferência da integridade referencial (as consultas devem retornar vazias)
SELECT * FROM MatriculasModalidades WHERE id_aluno = 2;
SELECT * FROM ExamesMedicos WHERE id_aluno = 2;
```

- **Cancelamento:** remove apenas a matrícula de Jeremias em Natação.
- **ON DELETE CASCADE:** ao excluir a aluna Camila Ribeiro (id 2), o banco apaga junto a matrícula e o exame dela. As duas consultas finais retornam vazias, ou seja, não ficam registros órfãos.
- **ON DELETE RESTRICT:** uma modalidade que tem instrutor não pode ser excluída. A aplicação mostra uma mensagem de erro em vez de apagar.

6. Seleções Complexas

As quatro consultas abaixo combinam junção de tabelas (JOIN) com filtros de condição (WHERE e HAVING). Os resultados foram obtidos executando as consultas sobre a carga inicial de dados.

Consulta 1: Ficha Geral de Alunos com Exame Apto (Junção de 5 tabelas)

Realiza o cruzamento de todas as tabelas do centro esportivo para emitir o relatório operacional de alunos ativos e autorizados pelo departamento médico.

```
SELECT
    a.nome_aluno AS Aluno,
    a.cpf AS CPF,
    m.nome_modalidade AS Modalidade,
    CONCAT('R$ ', FORMAT(m.preco_mensal, 2, 'pt_BR')) AS Mensalidade,
    i.nome_instrutor AS Instrutor_Responsavel,
    i.cref AS CREF,
    em.situacao AS Situacao_Medica,
    DATE_FORMAT(mm.data_adesao, '%d/%m/%Y') AS Data_Adesao
FROM Alunos a
INNER JOIN MatriculasModalidades mm ON a.id_aluno = mm.id_aluno
INNER JOIN Modalidades m ON mm.id_modalidade = m.id_modalidade
INNER JOIN Instrutores i ON m.id_modalidade = i.id_modalidade
INNER JOIN ExamesMedicos em ON a.id_aluno = em.id_aluno
WHERE em.situacao = 'Apto'
  AND m.preco_mensal >= 90.00
ORDER BY a.nome_aluno ASC, m.nome_modalidade ASC;
```

- **Junção:** INNER JOIN entre Alunos, MatriculasModalidades, Modalidades, Instrutores e ExamesMedicos. Só aparecem alunos que têm matrícula e exame.
- **Filtros:** WHERE situacao = 'Apto' e preco_mensal >= 90,00.
- **Ordenação:** por nome do aluno e depois por modalidade.

Resultado:

Aluno	Modalidade	Mensalidade	Instrutor	Situação	Adesão

Beatriz Lima	Pilates	R$ 130,00	Profª Fernanda Souza	Apto	15/05/2026
Camila Ribeiro	Crossfit	R$ 150,00	Profª Maria Clara Pastor	Apto	15/02/2026
Helena Vieira	Crossfit	R$ 150,00	Profª Maria Clara Pastor	Apto	20/01/2026
Helena Vieira	Musculação	R$ 90,00	Prof. Caio Polidoro	Apto	10/01/2026
Lucas Silva	Musculação	R$ 90,00	Prof. Caio Polidoro	Apto	10/04/2026
Lucas Silva	Natação	R$ 150,00	Prof. Marcelo Costa	Apto	12/04/2026

Consulta 2: Auditoria Médica Preventiva (Alunos Inaptos em Atividades Físicas)

Identifica quais alunos possuem matrícula ativa mas estão reprovados no laudo médico, listando a modalidade e o instrutor para bloqueio na catraca.

```
SELECT
    a.id_aluno,
    a.nome_aluno AS Aluno,
    em.situacao AS Status_Exame,
    DATE_FORMAT(em.data_avaliacao, '%d/%m/%Y') AS Data_Avaliacao_Medica,
    m.nome_modalidade AS Modalidade_Matriculada,
    i.nome_instrutor AS Instrutor_Avisado
FROM Alunos a
INNER JOIN ExamesMedicos em ON a.id_aluno = em.id_aluno
INNER JOIN MatriculasModalidades mm ON a.id_aluno = mm.id_aluno
INNER JOIN Modalidades m ON mm.id_modalidade = m.id_modalidade
INNER JOIN Instrutores i ON m.id_modalidade = i.id_modalidade
WHERE em.situacao = 'Inapto';
```

- **Junção:** INNER JOIN das mesmas 5 tabelas, começando por Alunos e ExamesMedicos.
- **Filtro:** WHERE situacao = 'Inapto', que seleciona só os alunos sem liberação médica.

Resultado:

Id	Aluno	Exame	Data da avaliação	Modalidade	Instrutor avisado
3	Jeremias Souza	Inapto	14/04/2026	Natação	Prof. Marcelo Costa

Consulta 3: Faturamento Estimado e Quantidade de Alunos por Modalidade

Mostra o faturamento do centro esportivo e a quantidade de alunos em cada modalidade.

```
SELECT
    m.id_modalidade,
```

```
    m.nome_modalidade AS Modalidade,
    i.nome_instrutor AS Instrutor,
    m.preco_mensal AS Valor_Unitario,
    COUNT(mm.id_aluno) AS Total_Alunos_Matriculados,
    SUM(m.preco_mensal) AS Receita_Mensal_Gerada
FROM Modalidades m
LEFT JOIN Instrutores i ON m.id_modalidade = i.id_modalidade
LEFT JOIN MatriculasModalidades mm ON m.id_modalidade = mm.id_modalidade
GROUP BY m.id_modalidade, m.nome_modalidade, i.nome_instrutor, m.preco_mensal
HAVING COUNT(mm.id_aluno) > 0
ORDER BY Receita_Mensal_Gerada DESC;
```

- **Junção:** LEFT JOIN a partir de Modalidades, para que todas as modalidades entrem no agrupamento, mesmo sem aluno.
- **Agregação:** GROUP BY por modalidade, com COUNT (número de alunos) e SUM (receita mensal).
- **Filtro sobre o grupo:** HAVING COUNT(mm.id_aluno) > 0 descarta modalidades sem alunos. Por isso o Boxe não aparece.
- **Ordenação:** da maior para a menor receita.

Resultado:

Modalidade	Instrutor	Valor unitário	Alunos	Receita mensal
Crossfit	Profª Maria Clara Pastor	R$ 150,00	2	R$ 300,00
Natação	Prof. Marcelo Costa	R$ 150,00	2	R$ 300,00
Musculação	Prof. Caio Polidoro	R$ 90,00	2	R$ 180,00
Pilates	Profª Fernanda Souza	R$ 130,00	1	R$ 130,00

Consulta 4: Alunos Multidisciplinares e Mensalidade Acumulada

Localiza alunos que praticam mais de um esporte e calcula o pacote financeiro total de cada um, ou seja, a soma das mensalidades de todas as modalidades em que está matriculado.

```
SELECT
    a.id_aluno,
    a.nome_aluno AS Aluno,
    COUNT(mm.id_modalidade) AS Qtd_Modalidades,
    GROUP_CONCAT(m.nome_modalidade SEPARATOR ', ') AS Modalidades_Cursadas,
    SUM(m.preco_mensal) AS Mensalidade_Total_Aluno
FROM Alunos a
INNER JOIN MatriculasModalidades mm ON a.id_aluno = mm.id_aluno
INNER JOIN Modalidades m ON mm.id_modalidade = m.id_modalidade
GROUP BY a.id_aluno, a.nome_aluno
HAVING COUNT(mm.id_modalidade) > 1
ORDER BY Mensalidade_Total_Aluno DESC;
```

- **Junção:** INNER JOIN entre Alunos, MatriculasModalidades e Modalidades.

- **Agregação:** GROUP BY por aluno, com COUNT (quantidade de modalidades), GROUP_CONCAT (nomes das modalidades em uma só coluna) e SUM (mensalidade total).
- **Filtro sobre o grupo:** HAVING COUNT(mm.id_modalidade) > 1 mantém só quem pratica mais de um esporte.

Resultado:

Id	Aluno	Qtd. modalidades	Modalidades cursadas	Mensalidade total
1	Helena Vieira	2	Musculação, Crossfit	R$ 240,00
4	Lucas Silva	2	Musculação, Natação	R$ 240,00

7. Conclusão

O trabalho implementou o sistema do centro esportivo desde a modelagem conceitual até o código. O DER e o modelo lógico definem as cinco tabelas e as relações 1:1, 1:N e N:N. O diagrama de classes e a arquitetura de persistência mostram como essas tabelas viram objetos na aplicação, com um adaptador objeto-relacional que concentra todo o SQL. O código implementa seleção, inserção, atualização e remoção, além de quatro seleções complexas com junções de até cinco tabelas e filtros com WHERE e HAVING.